

PROGRAMACIÓN II

Trabajo Práctico 6 – Colecciones

Alumno: Chiavón, Cristian

Repositorio GitHub: <https://github.com/cridvon/UTN-TUPaD-P2>

Caso Práctico 1

Se debe desarrollar un sistema de stock que permita gestionar productos en una tienda, controlando su disponibilidad, precios y categorías.

La información se modelará utilizando clases, colecciones dinámicas y enumeraciones en Java.

Clase Main

```
1 package tienda;
2
3 public class Tienda {
4
5     public static void main(String[] args) {
6
7         Inventario inventario = new Inventario();
8         Producto productoBuscado;
9
10        Producto fideo = new Producto("1", "Fideo", 1200.00, 5, CategoriaProducto.ALIMENTOS);
11        Producto monitor = new Producto("2", "Monitor", 230000.00, 2, CategoriaProducto.ELECTRONICA);
12        Producto gorra = new Producto("3", "Gorra", 15000.00, 4, CategoriaProducto.ROPA);
13        Producto mesa = new Producto("4", "Mesa", 120000.00, 1, CategoriaProducto.HOGAR);
14        Producto arroz = new Producto("5", "Arroz", 900.00, 3, CategoriaProducto.ALIMENTOS);
15
16        inventario.agregarProducto(fideo);
17        inventario.agregarProducto(monitor);
18        inventario.agregarProducto(gorra);
19        inventario.agregarProducto(mesa);
20        inventario.agregarProducto(arroz);
21
22        System.out.println("Listado de productos actuales -----");
23        inventario.listarProductos();
24
25        System.out.println("-----");
26        System.out.println("Se busca producto con id 1");
27        productoBuscado = inventario.buscarProductoPorId("1");
28        if (productoBuscado == null) {
29            System.out.println("No tenemos ese producto en la tienda");
30        } else {
31            System.out.println("Producto encontrado: " + productoBuscado);
32        }
33    }
34
35    System.out.println("-----");
36    System.out.println("Filtrar productos con categoria ALIMENTOS");
37    inventario.filtrarPorCategoria(CategoriaProducto.ALIMENTOS);
38
39    System.out.println("-----");
40    System.out.println("Se elimina producto con id 3");
41    inventario.eliminarProducto("3");
42
43    System.out.println("-----");
44    System.out.println("Listado de productos actuales -----");
45    inventario.listarProductos();
46
47    System.out.println("-----");
48    System.out.println("Se actualiza stock de mesa -----");
49    inventario.actualizarStock("4", 9);
50
51    System.out.println("-----");
52    System.out.println("Listado de productos actuales -----");
53    inventario.listarProductos();
54
55    System.out.println("-----");
56    System.out.println("Stock disponibles -----");
57    inventario.obtenerTotalStock();
58
59    System.out.println("-----");
60    System.out.println("Producto con mayor stock -----");
61    inventario.obtenerProductoConMayorStock();
62
63    System.out.println("-----");
64    System.out.println("Filtrar productos por rango de precio -----");
65    inventario.filtrarProductosPorPrecio(1000, 3000);
66
67    System.out.println("-----");
68    System.out.println("Categorías disponibles con sus descripciones -----");
69    inventario.mostrarCategoriasDisponibles();
70    }
71 }
72 }
```

Clase Producto

```
package tienda;

public class Producto {

    private String id;
    private String nombre;
    private double precio;
    private int cantidad;
    private CategoriaProducto categoria;

    public Producto(String id, String nombre, double precio, int cantidad, CategoriaProducto categoria) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.cantidad = cantidad;
        this.categoria = categoria;
    }

    public void mostrarInfo() {
        System.out.println(this);
    }

    public String getId() {
        return this.id;
    }

    public String getNombre() {
        return this.nombre;
    }

    public CategoriaProducto getCategoria() {
        return categoria;
    }

    public void setStock(int cantidad) {
        this.cantidad = cantidad;
        System.out.println("Stock actualizado");
    }

    public int getStock() {
        return this.cantidad;
    }

    public double getPrecio() {
        return this.precio;
    }

    @Override
    public String toString() {
        return "Producto(" + "id=" + id + ", nombre=" + nombre + ", precio=" + precio + ", cantidad=" + cantidad + ", categoria=" + categoria + ')';
    }
}
```

Enum CategoriaProductos

```
1 package tienda;
2
3 public enum CategoriaProducto {
4
5     ALIMENTOS("Productos comestibles"),
6     ELECTRONICA("Dispositivos electrónicos"),
7     ROPA("Prendas de vestir"),
8     HOGAR("Artículos para el hogar");
9
10    private final String descripcion;
11
12    CategoriaProducto(String descripcion) {
13        this.descripcion = descripcion;
14    }
15
16    public String getDescripcion() {
17        return descripcion;
18    }
19
20 }
21
```

Clase Inventario

```

1  package tienda;
2
3  import java.util.ArrayList;
4  import java.util.Iterator;
5
6  public class Inventario {
7
8      ArrayList<Producto> productos = new ArrayList<>();
9
10     public void agregarProducto(Producto p) {
11         productos.add(p);
12     }
13
14     public void listarProductos() {
15         for (Producto producto : productos) {
16             System.out.println(producto);
17         }
18     }
19
20     public Producto buscarProductoPorId(String id) {
21         Producto productoEncontrado = null;
22         Iterator<Producto> it = this.productos.iterator();
23         while (it.hasNext() && productoEncontrado == null) {
24             Producto next = it.next();
25             if (next.getId().equalsIgnoreCase(id)) {
26                 productoEncontrado = next;
27             }
28         }
29         return productoEncontrado;
30     }
31
32     public void eliminarProducto(String id) {
33         Producto productoABorrar = buscarProductoPorId(id);
34         productos.remove(productoABorrar);
35     }
36
37     public void actualizarStock(String id, int nuevaCantidad) {
38         Producto buscado = buscarProductoPorId(id);
39         if (buscado == null) {
40             System.out.println("No se puede actualizar un producto que no existe");
41         } else {
42             buscado.setStock(nuevaCantidad);
43         }
44     }
45
46     public void filtrarPorCategoria(CategoriaProducto categoria) {
47         ArrayList<Producto> productosFiltrados = new ArrayList<>();
48         for (Producto producto : productos) {
49             if (producto.getCategoria().equals(categoria)) {
50                 productosFiltrados.add(producto);
51             }
52         }
53
54         System.out.println("Productos con la categoria " + categoria + " : ");
55         for (Producto productosFiltrado : productosFiltrados) {
56             System.out.println(productosFiltrado);
57         }
58     }
59
60     public void obtenerTotalStock() {
61         for (Producto producto : productos) {
62             System.out.println(producto.getNombre() + " : " + producto.getStock());
63         }
64     }
65
66     public void obtenerProductoConMayorStock() {
67         Producto prodStockMax = null;
68         int stockMax = 0;
69
70         for (Producto producto : productos) {
71             if (producto.getStock() > stockMax) {
72                 prodStockMax = producto;
73                 stockMax = producto.getStock();
74             }
75         }
76
77         System.out.println(prodStockMax);
78     }
79
80 }

```

```

81
82 public void filtrarProductosPorPrecio(double min, double max) {
83     ArrayList<Producto> productosFiltrados = new ArrayList<>();
84     for (Producto producto : productos) {
85         if (producto.getPrecio() >= min && producto.getPrecio() <= max) {
86             productosFiltrados.add(producto);
87         }
88     }
89
90     System.out.println("Productos con precio entre " + min + " y " + max + " : ");
91     for (Producto productoFiltrado : productosFiltrados) {
92         System.out.println(productoFiltrado);
93     }
94 }
95
96 public void mostrarCategoriasDisponibles() {
97     for (Producto producto : productos) {
98         System.out.println(producto.getNombre() + " - " + producto.getCategoria().getDescripcion());
99     }
100 }
101
102 }
103

```

Resultados

```

Run:
Listado de productos actuales -----
Producto{id=1, nombre=Fideo, precio=1200.0, cantidad=5, categoria=ALIMENTOS}
Producto{id=2, nombre=Monitor, precio=230000.0, cantidad=2, categoria=ELECTRONICA}
Producto{id=3, nombre=Gorra, precio=15000.0, cantidad=4, categoria=ROPA}
Producto{id=4, nombre=Mesa, precio=120000.0, cantidad=1, categoria=HOGAR}
Producto{id=5, nombre=Arroz, precio=900.0, cantidad=3, categoria=ALIMENTOS}
-----
Se busca producto con id 1
Producto encontrado: Producto{id=1, nombre=Fideo, precio=1200.0, cantidad=5, categoria=ALIMENTOS}
-----
Filtrar productos con categoria ALIMENTOS
Productos con la categoria ALIMENTOS :
Producto{id=1, nombre=Fideo, precio=1200.0, cantidad=5, categoria=ALIMENTOS}
Producto{id=5, nombre=Arroz, precio=900.0, cantidad=3, categoria=ALIMENTOS}
-----
Se elimina producto con id 3
-----
Listado de productos actuales -----
Producto{id=1, nombre=Fideo, precio=1200.0, cantidad=5, categoria=ALIMENTOS}
Producto{id=2, nombre=Monitor, precio=230000.0, cantidad=2, categoria=ELECTRONICA}
Producto{id=4, nombre=Mesa, precio=120000.0, cantidad=1, categoria=HOGAR}
Producto{id=5, nombre=Arroz, precio=900.0, cantidad=3, categoria=ALIMENTOS}
-----
Se actualiza stock de mesa -----
Stock actualizado
-----
Listado de productos actuales -----
Producto{id=1, nombre=Fideo, precio=1200.0, cantidad=5, categoria=ALIMENTOS}
Producto{id=2, nombre=Monitor, precio=230000.0, cantidad=2, categoria=ELECTRONICA}
Producto{id=4, nombre=Mesa, precio=120000.0, cantidad=9, categoria=HOGAR}
Producto{id=5, nombre=Arroz, precio=900.0, cantidad=3, categoria=ALIMENTOS}
-----
Stock disponibles -----
Fideo : 5
Monitor : 2
Mesa : 9
Arroz : 3
-----
Producto con mayor stock -----
Producto{id=4, nombre=Mesa, precio=120000.0, cantidad=9, categoria=HOGAR}
-----
Filtrar productos por rango de precio -----
Productos con precio entre 1000.0 y 3000.0 :
Producto{id=1, nombre=Fideo, precio=1200.0, cantidad=5, categoria=ALIMENTOS}
-----
Categorias disponibles con sus descripciones -----
Fideo - Productos comestibles
Monitor - Dispositivos electronicos
Mesa - Articulos para el hogar
Arroz - Productos comestibles
BUILD SUCCESSFUL (total time: 0 seconds)

```

Caso Práctico 2

Se debe desarrollar un sistema para gestionar una biblioteca, en la cual se registren los libros disponibles y sus autores.

La relación central es de composición 1 a N: una Biblioteca contiene múltiples Libros, y cada Libro pertenece obligatoriamente a una Biblioteca.

Si la Biblioteca se elimina, también se eliminan sus Libros.

Clase Main

```
package biblioteca;

public class Gestor_Biblioteca {

    public static void main(String[] args) {
        Biblioteca biblioteca = new Biblioteca();

        Autor marquez = new Autor("1", "Gabriel García Márquez", "Colombiano");
        Autor cortazar = new Autor("2", "Julio Cortazar", "Argentino");
        Autor borges = new Autor("3", "Jorge Lui Borges", "Argentino");

        biblioteca.agregarLibro("978-9500745093", "Cien años de soledad", 1967, marquez);
        biblioteca.agregarLibro("950-0708167", "El amor en los tiempos del cólera", 1985, marquez);
        biblioteca.agregarLibro("978-8402076939", "Crónica de una muerte anunciada", 1981, marquez);
        biblioteca.agregarLibro("84-487-0403-7", "Rayuela", 1985, cortazar);
        biblioteca.agregarLibro("978-8420633121", "Ficciones", 1944, borges);

        System.out.println("Listado de libros actuales -----");
        biblioteca.listarLibros();

        System.out.println("-----");
        System.out.println("Se busca libro por ISBN = 84-487-0403-7 -----");
        Libro libroBuscado = biblioteca.buscarLibroPorIsbn("84-487-0403-7");
        if (libroBuscado == null) {
            System.out.println("No existe ese libro en la biblioteca");
        } else {
            System.out.println("Libro encontrado: ");
            System.out.println(libroBuscado);
        }

        System.out.println("-----");
        System.out.println("Filtrimos Libros según fecha -----");
        biblioteca.filtrarLibrosPorAño(1985);

        System.out.println("-----");
        System.out.println("Se elimina libro por su ISBN : 84-487-0403-7");
        biblioteca.eliminarLibro("84-487-0403-7");

        System.out.println("-----");
        System.out.println("Listado de libros actuales -----");
        biblioteca.listarLibros();

        System.out.println("-----");
        biblioteca.obtenerCantidadLibros();

        System.out.println("-----");
        biblioteca.mostrarAutoresDisponibles();
    }
}
```

**Clase Autor**

```
package biblioteca;

public class Autor {

    private String id;
    private String nombre;
    private String nacionalidad;

    public Autor(String id, String nombre, String nacionalidad) {
        this.id = id;
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }

    public void mostrarInfo() {
        System.out.println(this);
    }

    public String getNombre() {
        return this.nombre;
    }

    @Override
    public String toString() {
        return "Autor{" + "id=" + id + ", nombre=" + nombre + ", nacionalidad=" + nacionalidad + '}';
    }

}
```

Clase Libro

```
package biblioteca;

public class Libro {

    private String isbn;
    private String titulo;
    private int anioPublicacion;
    private Autor autor;

    public Libro(String isbn, String titulo, int anioPublicacion, Autor autor) {
        this.isbn = isbn;
        this.titulo = titulo;
        this.anioPublicacion = anioPublicacion;
        this.autor = autor;
    }

    public String getIsbn() {
        return this.isbn;
    }

    public int getAnio() {
        return this.anioPublicacion;
    }

    public Autor getAutor() {
        return this.autor;
    }

    @Override
    public String toString() {
        return "Libro{" + "isbn=" + isbn + ", titulo=" + titulo + ", anioPublicacion=" + anioPublicacion + ", autor=" + autor + '}';
    }

}
```

Clase Biblioteca

```

package biblioteca;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Biblioteca {

    private String nombre;
    private List<Libro> libros = new ArrayList<>();

    public void agregarLibro(String isbn, String titulo, int anioPublicacion, Autor autor) {
        Libro libro = new Libro(isbn, titulo, anioPublicacion, autor);
        libros.add(libro);
    }

    public void listarLibros() {
        for (Libro libro : libros) {
            System.out.println(libro);
        }
    }

    public Libro buscarLibroPorIsbn(String isbn) {
        Libro libroEncontrado = null;
        Iterator<Libro> it = this.libros.iterator();
        while (it.hasNext() && libroEncontrado == null) {
            Libro next = it.next();
            if (next.getIsbn().equalsIgnoreCase(isbn)) {
                libroEncontrado = next;
            }
        }
        return libroEncontrado;
    }

    public void filtrarLibrosPorAnio(int anio) {
        ArrayList<Libro> librosFiltrados = new ArrayList<>();
        for (Libro libro : libros) {
            if (libro.getAnio() == anio) {
                librosFiltrados.add(libro);
            }
        }

        System.out.println("Libros publicados en " + anio + " : ");
        for (Libro librosFiltrado : librosFiltrados) {
            System.out.println(librosFiltrado);
        }
    }

    public void eliminarLibro(String isbn) {
        Libro libroBuscado = buscarLibroPorIsbn(isbn);
        if (libroBuscado == null) {
            System.out.println("No existe el libro");
        } else {
            libros.remove(libroBuscado);
            System.out.println("Libro Borrado");
        }
    }

    public void obtenerCantidadLibros() {
        System.out.println("Cantidad de libros en Biblioteca: " + libros.size());
    }

    public void mostrarAutoresDisponibles() {
        System.out.println("Autores disponibles");

        ArrayList<Autor> autoresDisponibles = new ArrayList<>();

        for (Libro libro : libros) {
            Autor autor = libro.getAutor();
            if (!autoresDisponibles.contains(author)) {
                autoresDisponibles.add(author);
            }
        }

        for (Autor autor : autoresDisponibles) {
            System.out.println(autor.getNombre());
        }
    }
}

```


Resultados

```

fun:
Listado de libros actuales -----
Libro{isbn=978-9500745093, titulo=Cien años de soledad, añoPublicacion=1967, autor=Autor{id=1, nombre=Gabriel García Márquez, nacionalidad=Colombiano}}
Libro{isbn=950-0708167, titulo=El amor en los tiempos del cólera, añoPublicacion=1985, autor=Autor{id=1, nombre=Gabriel García Márquez, nacionalidad=Colombiano}}
Libro{isbn=978-8402076939, titulo=Crónica de una muerte anunciada, añoPublicacion=1981, autor=Autor{id=1, nombre=Gabriel García Márquez, nacionalidad=Colombiano}}
Libro{isbn=84-487-0403-7, titulo=Rayuela, añoPublicacion=1985, autor=Autor{id=2, nombre=Julio Cortázar, nacionalidad=Argentino}}
Libro{isbn=978-8420639121, titulo=Ficciones, añoPublicacion=1944, autor=Autor{id=3, nombre=Jorge Luis Borges, nacionalidad=Argentino}}
-----
Se busca libro por ISBN = 84-487-0403-7 -----
Libro encontrado:
Libro{isbn=84-487-0403-7, titulo=Rayuela, añoPublicacion=1985, autor=Autor{id=2, nombre=Julio Cortázar, nacionalidad=Argentino}}
-----
Filtramos Libros según fecha -----
Libros publicados en 1985 :
Libro{isbn=950-0708167, titulo=El amor en los tiempos del cólera, añoPublicacion=1985, autor=Autor{id=1, nombre=Gabriel García Márquez, nacionalidad=Colombiano}}
Libro{isbn=84-487-0403-7, titulo=Rayuela, añoPublicacion=1985, autor=Autor{id=2, nombre=Julio Cortázar, nacionalidad=Argentino}}
-----
Se elimina libro por su ISBN : 84-487-0403-7
Libro Borrado
-----
Listado de libros actuales -----
Libro{isbn=978-9500745093, titulo=Cien años de soledad, añoPublicacion=1967, autor=Autor{id=1, nombre=Gabriel García Márquez, nacionalidad=Colombiano}}
Libro{isbn=950-0708167, titulo=El amor en los tiempos del cólera, añoPublicacion=1985, autor=Autor{id=1, nombre=Gabriel García Márquez, nacionalidad=Colombiano}}
Libro{isbn=978-8402076939, titulo=Crónica de una muerte anunciada, añoPublicacion=1981, autor=Autor{id=1, nombre=Gabriel García Márquez, nacionalidad=Colombiano}}
Libro{isbn=978-8420639121, titulo=Ficciones, añoPublicacion=1944, autor=Autor{id=3, nombre=Jorge Luis Borges, nacionalidad=Argentino}}
-----
Cantidad de libros en Biblioteca: 4
-----
Autores disponibles
Gabriel García Márquez
Jorge Luis Borges
BUILD SUCCESSFUL (total time: 0 seconds)

```


Caso Práctico 3

Se debe modelar un sistema académico donde un Profesor dicta muchos Cursos y cada Curso tiene exactamente un Profesor responsable.

Clase Main

```

1 package Academia;
2
3 public class Academia {
4
5     public static void main(String[] args) {
6         Universidad universidad = new Universidad();
7
8         Profesor facu = new Profesor("1", "Facundo", "Computación");
9         Profesor cris = new Profesor("2", "Cristian", "Matemática");
10        Profesor ana = new Profesor("3", "Anahí", "RRHH");
11
12        Curso programacion = new Curso("1-0020", "Programación 1");
13        Curso organizacion = new Curso("3-0010", "Organización Empresarial");
14        Curso ecuaciones = new Curso("2-0220", "Ecuaciones Lineales");
15        Curso bbdd = new Curso("1-0025", "Base de Datos");
16        Curso liquidaciones = new Curso("3-0005", "Liquidaciones");
17
18        universidad.agregarProfesor(facu);
19        universidad.agregarProfesor(cris);
20        universidad.agregarProfesor(ana);
21
22        universidad.agregarCurso(programacion);
23        universidad.agregarCurso(organizacion);
24        universidad.agregarCurso(ecuaciones);
25        universidad.agregarCurso(bbdd);
26        universidad.agregarCurso(liquidaciones);
27
28        System.out.println("Lista de Profesores -----");
29        universidad.listarProfesores();
30
31        System.out.println("Lista de Cursos -----");
32        universidad.listarCursos();
33
34        System.out.println("-----");
35        System.out.println("Se asignan Profesores a Cursos ");
36        universidad.asignarProfesorACurso("1-0020", "1");
37        universidad.asignarProfesorACurso("3-0010", "3");
38        universidad.asignarProfesorACurso("2-0220", "2");
39        universidad.asignarProfesorACurso("1-0025", "1");
40        universidad.asignarProfesorACurso("3-0005", "3");
41        System.out.println("-----");
42
43        System.out.println("Lista de Profesores -----");
44        universidad.listarProfesores();
45
46        System.out.println("Lista de Cursos -----");
47        universidad.listarCursos();
48
49        System.out.println("-----");
50        System.out.println("Se cambia curso 3-0010 a Profesor 2");
51        universidad.asignarProfesorACurso("3-0010", "2");
52        System.out.println("-----");
53
54        System.out.println("Lista de Profesores -----");
55        universidad.listarProfesores();
56
57        System.out.println("Lista de Cursos -----");
58        universidad.listarCursos();
59
60        System.out.println("-----");
61        System.out.println("Se elimina Profesor 2");
62        universidad.eliminarProfesor("2");
63        System.out.println("-----");
64
65        System.out.println("Lista de Profesores -----");
66        universidad.listarProfesores();
67
68        System.out.println("Lista de Cursos -----");
69        universidad.listarCursos();
70
71    }
72
73 }

```

```
package Academia;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Universidad {

    private String nombre;
    private List<Profesor> profesores = new ArrayList<>();
    private List<Curso> cursos = new ArrayList<>();

    public void agregarProfesor(Profesor p) {
        if (p != null) {
            profesores.add(p);
        } else {
            System.out.println("Debe proporcionar un valor válido");
        }
    }

    public void agregarCurso(Curso c) {
        if (c != null) {
            cursos.add(c);
        } else {
            System.out.println("Debe proporcionar un valor válido");
        }
    }

    // Usa setProfesor del curso
    public void asignarProfesorACurso(String codigoCurso, String idProfesor) {
        Profesor profesor = buscarProfesorPorId(idProfesor);
        Curso curso = buscarCursoPorCodigo(codigoCurso);

        Profesor profeViejo = curso.getProfesor();
        if (profeViejo != null) {
            profeViejo.eliminarCurso(curso);
        }

        curso.setProfesor(profesor);
    }

    public void listarProfesores() {
        for (Profesor profesor : profesores) {
            System.out.println(profesor);
        }
    }

    public void listarCursos() {
        for (Curso curso : cursos) {
            System.out.println(curso);
        }
    }
}
```

```
public Profesor buscarProfesorPorId(String id) {
    Profesor profesorBuscado = null;
    Iterator<Profesor> it = this.profesores.iterator();
    while (it.hasNext() && profesorBuscado == null) {
        Profesor next = it.next();
        if (next.getId().equalsIgnoreCase(id)) {
            profesorBuscado = next;
        }
    }
    return profesorBuscado;
}

public Curso buscarCursoPorCodigo(String codigo) {
    Curso cursoBuscado = null;
    Iterator<Curso> it = this.cursos.iterator();
    while (it.hasNext() && cursoBuscado == null) {
        Curso next = it.next();
        if (next.getCodigo().equalsIgnoreCase(codigo)) {
            cursoBuscado = next;
        }
    }
    return cursoBuscado;
}

//Debe romper la relación con su profesor si la hubiera.
public void eliminarCurso(String codigo) {
    Curso cursoBuscado = buscarCursoPorCodigo(codigo);
    Profesor profesor = cursoBuscado.getProfesor();

    if (profesor != null) {
        profesor.eliminarCurso(cursoBuscado);
    }
}

//Antes de remover, dejar null los cursos que dictaba.
public void eliminarProfesor(String id) {
    Profesor profesor = buscarProfesorPorId(id);
    for (Curso curso : cursos) {
        if (curso.getProfesor() == profesor) {
            curso.eliminarProfesor();
        }
    }
    profesores.remove(profesor);
}
```

Clase Profesor

```

package Academia;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Profesor {

    private String id;
    private String nombre;
    private String especialidad;
    private List<Curso> cursos = new ArrayList<>();

    public Profesor(String id, String nombre, String especialidad) {
        this.id = id;
        this.nombre = nombre;
        this.especialidad = especialidad;
    }

    public String getNombre() {
        return this.nombre;
    }

    public String getId() {
        return this.id;
    }

    public Profesor getProfesor() {
        return this;
    }

    public void agregarCurso(Curso c) {
        if (!cursos.contains(c)) {
            cursos.add(c);
            c.setProfesor(this);
        } else {
            System.out.println("El profesor ya tiene asignado ese curso");
        }
    }

    public void eliminarCurso(Curso c) {
        if (cursos.contains(c)) {
            cursos.remove(c);
            if (c.getProfesor() == this) {
                c.eliminarProfesor();
            }
        } else {
            System.out.println("El profesor no tiene asignado ese curso");
        }
    }

    public Curso buscarCurso(Curso c) {
        Curso cursoBuscado = null;
        Iterator<Curso> it = this.cursos.iterator();
        while (it.hasNext() && cursoBuscado == null) {
            Curso next = it.next();
            if (next == c) {
                cursoBuscado = next;
            }
        }
        return cursoBuscado;
    }

    public void listarCursos() {
        for (Curso curso : cursos) {
            System.out.println(curso.getCodigo() + " - " + curso.getNombre());
        }
    }

    public void mostrarInfo() {
        System.out.println(this);
    }

    @Override
    public String toString() {
        return "Profesor{" + "id=" + id + ", nombre=" + nombre + ", especialidad=" + especialidad + ", cursos=" + cursos.size() + '}';
    }
}

```

Clase Curso

```
package Academia;

public class Curso {

    private String codigo;
    private String nombre;
    private Profesor profesor;

    public Curso(String codigo, String nombre) {
        this.codigo = codigo;
        this.nombre = nombre;
    }

    public String getNombre() {
        return this.nombre;
    }

    public String getCodigo() {
        return this.codigo;
    }

    public void setProfesor(Profesor p) {
        this.profesor = p;
        Curso curso = p.buscarCurso(this);
        if (curso == null) {
            profesor.agregarCurso(this);
        }
    }

    public Profesor getProfesor() {
        return this.profesor;
    }

    public void eliminarProfesor() {
        this.profesor = null;
    }

    public Curso getCurso() {
        return this;
    }

    public void mostrarInfo() {
        System.out.println(this);
    }

    public String mostrarProfesor() {
        if (this.profesor == null) {
            return "";
        } else {
            return ", profesor= " + this.profesor.getNombre();
        }
    }

    @Override
    public String toString() {
        return "Curso(" + "codigo=" + codigo + ", nombre=" + nombre + mostrarProfesor() + ")";
    }
}
```

Resultados

```

run:
Lista de Profesores -----
Profesor{id=1, nombre=Facundo, especialidad=Computaci n, cursos=0}
Profesor{id=2, nombre=Cristian, especialidad=Matem tica, cursos=0}
Profesor{id=3, nombre=Anah , especialidad=RRHH, cursos=0}
Lista de Cursos -----
Curso{codigo=1-0020, nombre=Programaci n 1}
Curso{codigo=3-0010, nombre=Organizaci n Empresarial}
Curso{codigo=2-0220, nombre=Ecuaciones Lineales}
Curso{codigo=1-0025, nombre=Base de Datos}
Curso{codigo=3-0005, nombre=Liquidaciones}
-----
Se asignan Profesores a Cursos
-----
Lista de Profesores -----
Profesor{id=1, nombre=Facundo, especialidad=Computaci n, cursos=2}
Profesor{id=2, nombre=Cristian, especialidad=Matem tica, cursos=1}
Profesor{id=3, nombre=Anah , especialidad=RRHH, cursos=2}
Lista de Cursos -----
Curso{codigo=1-0020, nombre=Programaci n 1, profesor= Facundo}
Curso{codigo=3-0010, nombre=Organizaci n Empresarial, profesor= Anah }
Curso{codigo=2-0220, nombre=Ecuaciones Lineales, profesor= Cristian}
Curso{codigo=1-0025, nombre=Base de Datos, profesor= Facundo}
Curso{codigo=3-0005, nombre=Liquidaciones, profesor= Anah }
-----
Se cambia curso 3-0010 a Profesor 2
-----
Lista de Profesores -----
Profesor{id=1, nombre=Facundo, especialidad=Computaci n, cursos=2}
Profesor{id=2, nombre=Cristian, especialidad=Matem tica, cursos=2}
Profesor{id=3, nombre=Anah , especialidad=RRHH, cursos=1}
Lista de Cursos -----
Curso{codigo=1-0020, nombre=Programaci n 1, profesor= Facundo}
Curso{codigo=3-0010, nombre=Organizaci n Empresarial, profesor= Cristian}
Curso{codigo=2-0220, nombre=Ecuaciones Lineales, profesor= Cristian}
Curso{codigo=1-0025, nombre=Base de Datos, profesor= Facundo}
Curso{codigo=3-0005, nombre=Liquidaciones, profesor= Anah }
-----
Se elimina Profesor 2
-----
Lista de Profesores -----
Profesor{id=1, nombre=Facundo, especialidad=Computaci n, cursos=2}
Profesor{id=3, nombre=Anah , especialidad=RRHH, cursos=1}
Lista de Cursos -----
Curso{codigo=1-0020, nombre=Programaci n 1, profesor= Facundo}
Curso{codigo=3-0010, nombre=Organizaci n Empresarial}
Curso{codigo=2-0220, nombre=Ecuaciones Lineales}
Curso{codigo=1-0025, nombre=Base de Datos, profesor= Facundo}
Curso{codigo=3-0005, nombre=Liquidaciones, profesor= Anah }
BUILD SUCCESSFUL (total time: 0 seconds)

```